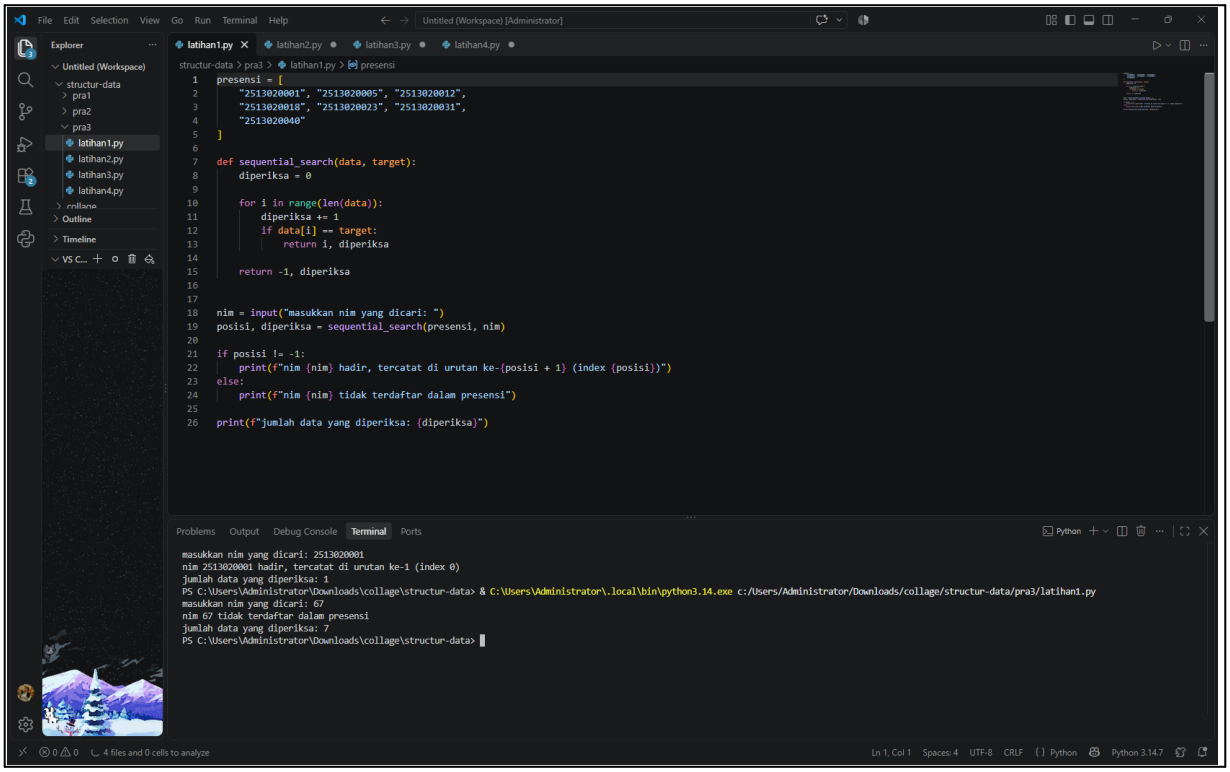
Prak 3 Struktur Data

Searching: Sequential Search (Tanpa dan Dengan Sentinel) dan Binary Search

Dosen Pengampu: Dr. Made Ayu Dusea Widyadara, M.Kom
Nama: Estu Lambang Wicaksono
NPM: 2513030136

Latihan 1 - Sequential Search Dasar



Gambar 1. Screenshot Latihan 1 (latihan1.py) - hasil eksekusi di terminal VS Code

Pertanyaan Reflektif

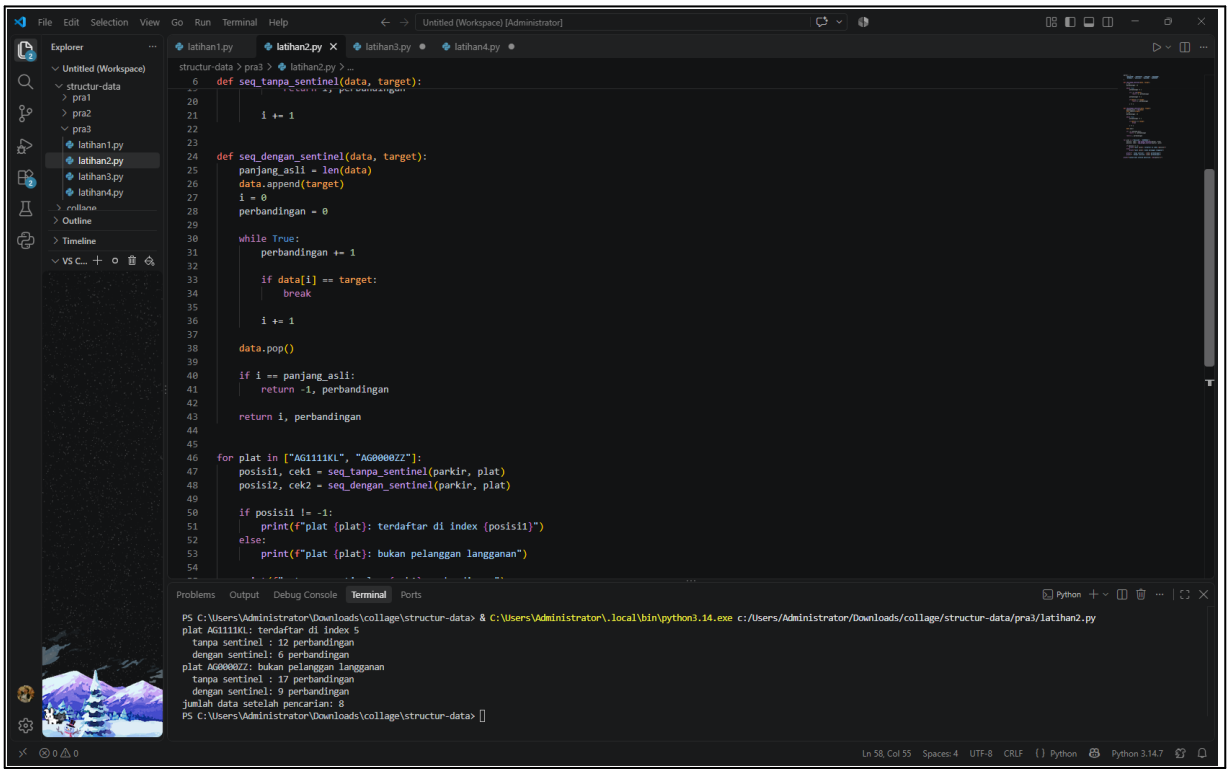
1. Mengapa jumlah data yang diperiksa selalu sama dengan panjang array ketika NIM tidak ditemukan?

Sequential search hanya bisa menyimpulkan bahwa sebuah NIM tidak ada setelah seluruh elemen array diperiksa. Data tidak terurut, sehingga tidak ada kondisi yang memungkinkan pencarian berhenti lebih awal. Perulangan baru selesai setelah elemen terakhir dibandingkan, sehingga jumlah pemeriksaan sama dengan n (pada program ini 7).

2. Mengapa NIM disimpan sebagai string, bukan integer?

NIM adalah identitas, bukan besaran yang dihitung, sehingga tidak pernah dijumlahkan atau dikali. Jika disimpan sebagai integer, angka nol di depan akan hilang. Selain itu, fungsi *input()* selalu menghasilkan string, sehingga perbandingan *data[i] == target* bisa langsung dilakukan. Bila array berisi integer sedangkan input berupa string, perbandingannya selalu False karena tipe datanya berbeda.

Latihan 2 - Sequential Search Tanpa vs Dengan Sentinel



Gambar 2. Screenshot Latihan 2 (latihan2.py) - hasil eksekusi di terminal VS Code

Pertanyaan Reflektif

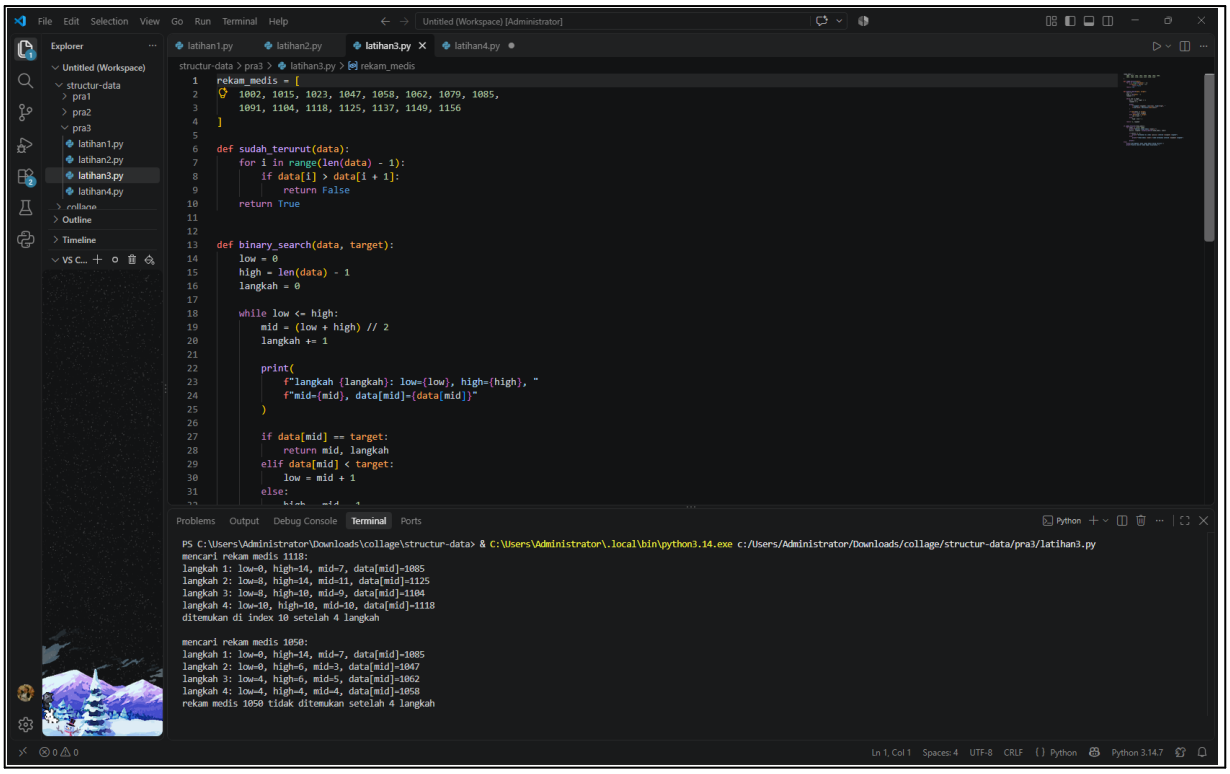
1. Apa yang terjadi pada data parkir jika lupa menulis data.pop() di fungsi sentinel, lalu pencarian dijalankan berkali-kali?

Setiap pemanggilan fungsi menambahkan satu elemen target ke ujung array dan tidak pernah dihapus, sehingga data parkir bertambah terus (8, 9, 10, dan seterusnya) dan tercemar oleh nilai yang bukan data asli. Akibatnya hasil pencarian menjadi salah. Contohnya, setelah AG0000ZZ dicari sekali, plat itu tertinggal di index 8. Pada pencarian berikutnya panjang data dihitung 9, sehingga AG0000ZZ dianggap ditemukan di index 8 padahal bukan pelanggan langganan. Selain itu, memori yang dipakai ikut membengkak.

2. Jika data berisi n elemen dan key tidak ditemukan, rumuskan jumlah perbandingan untuk masing-masing metode.

Tanpa sentinel: setiap putaran melakukan 2 perbandingan (cek batas dan cek nilai) sebanyak n kali, ditambah 1 cek batas terakhir ketika i = n, sehingga totalnya **2n + 1** (untuk n = 8 hasilnya 17). Dengan sentinel: setiap putaran hanya 1 perbandingan dan berhenti di posisi sentinel (putaran ke-(n + 1)), sehingga totalnya **n + 1** (untuk n = 8 hasilnya 9). Jadi sentinel menghemat sekitar setengah jumlah perbandingan.

Latihan 3 - Binary Search dengan Penelusuran Langkah



Gambar 3. Screenshot Latihan 3 (latihan3.py) - hasil eksekusi di terminal VS Code

Pertanyaan Reflektif

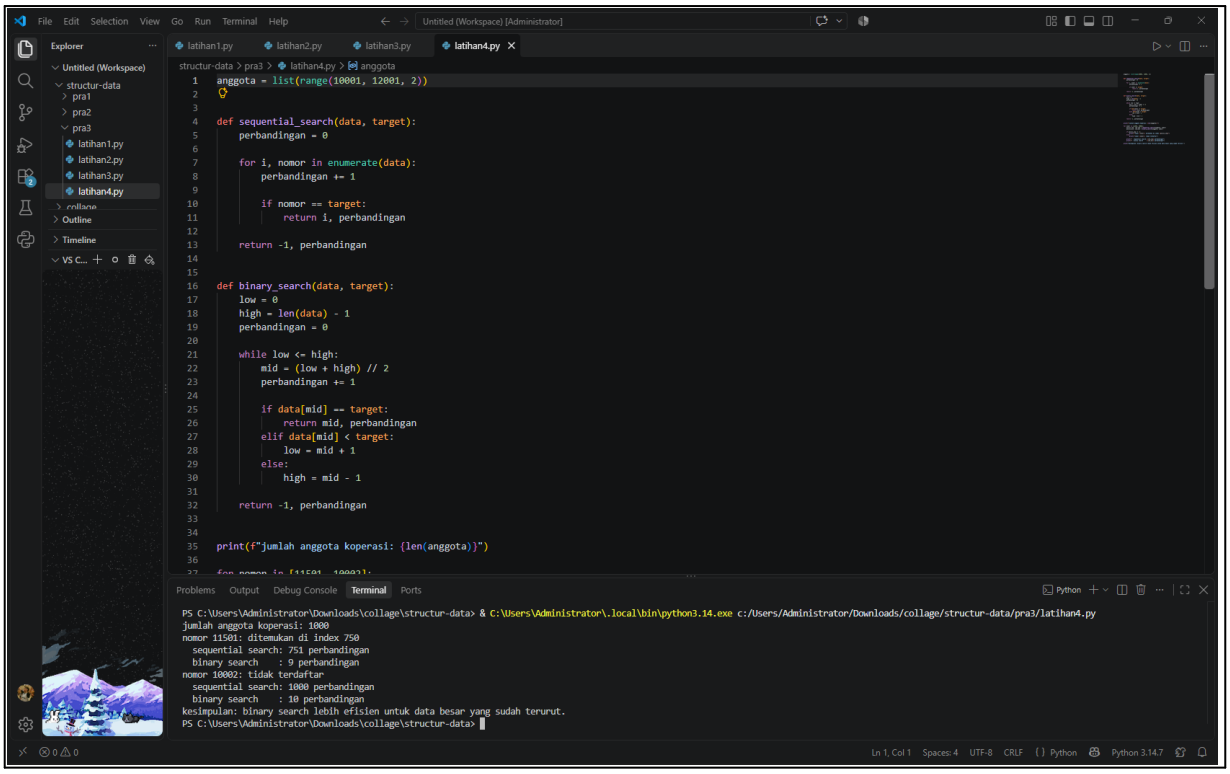
1. Pada pencarian 1050, perhatikan nilai low dan high setelah langkah 4. Mengapa perulangan berhenti di sana?

Pada langkah 4 nilainya low = 4, high = 4, mid = 4 dan data[4] = 1058. Karena 1058 lebih besar dari 1050, program menjalankan *high = mid - 1* sehingga high menjadi 3, sedangkan low tetap 4. Syarat perulangan *low <= high* menjadi salah (4 > 3) karena rentang pencarian sudah kosong, tidak ada lagi elemen yang mungkin bernilai 1050. Perulangan berhenti dan program menyimpulkan 1050 tidak ada di dalam array.

2. Ubah satu angka di rekam_medis agar urutannya rusak (mis. tukar 1047 dan 1085). Apa yang ditampilkan programmu, dan mengapa pemeriksaan ini penting?

Jika 1047 dan 1085 ditukar, fungsi *sudah_terurut()* menemukan data[3] = 1085 lebih besar dari data[4] = 1058 dan mengembalikan False. Program kemudian menampilkan peringatan bahwa data belum terurut dan tidak menjalankan binary search. Pemeriksaan ini penting karena binary search membuang setengah rentang berdasarkan asumsi data terurut. Pada data yang rusak, bagian yang berisi target bisa ikut terbuang. Contohnya mencari 1085 pada data yang sudah ditukar: pada langkah 1 mid = 7 bernilai 1047 yang lebih kecil dari 1085, sehingga low = 8 dan index 3 yang sebenarnya berisi 1085 tidak pernah diperiksa. Hasilnya program melaporkan tidak ditemukan padahal datanya ada. Dengan pemeriksaan ini, hasil yang salah seperti itu dapat dicegah.

Latihan 4 - Studi Kasus: Memilih Algoritma Pencarian



Gambar 4. Screenshot Latihan 4 (latihan4.py) - hasil eksekusi di terminal VS Code

Pertanyaan Reflektif

1. Contoh Soal memakai sequential search, sedangkan Soal Latihan cocok memakai binary search. Apa perbedaan kondisi data dan kebutuhan di kedua kasus?

Pada Contoh Soal (transaksi QRIS), data tidak terurut, ukurannya kecil, dan yang dicari adalah semua kemunculan sebuah nominal (bisa lebih dari satu). Kebutuhan seperti ini mengharuskan seluruh data dipindai, dan binary search tidak dapat dipakai karena data tidak terurut. Pada Soal Latihan (koperasi), data sudah terurut, jumlahnya besar (1000), nomor anggotanya unik, dan yang dicari hanya satu nomor. Keterurutan ini dimanfaatkan binary search untuk membuang setengah data di setiap langkah, sehingga cukup sekitar 10 perbandingan dibanding 1000 perbandingan pada sequential search.

2. Jika koperasi hanya mencari satu nomor sekali saja, tetapi datanya belum terurut, apakah tetap layak mengurutkan data dulu lalu memakai binary search?

Tidak layak. Mengurutkan data memerlukan waktu sekitar O(n log n), sedangkan sequential search untuk satu pencarian hanya O(n). Untuk n = 1000, pengurutan membutuhkan kira-kira 10.000 operasi, sedangkan sequential search paling banyak 1000 perbandingan. Biaya pengurutan lebih besar daripada penghematan yang didapat dari binary search pada satu kali pencarian. Mengurutkan baru menguntungkan jika pencarian dilakukan berkali-kali pada data yang sama, karena biaya pengurutan hanya dibayar sekali dan dibagi ke banyak pencarian.